FINAL YEAR PROJECT · BEng Software & Electronic Engineering · 2025/2026

Smart System Performance Dashboard

Real-time Hardware Monitoring with ML-Powered Fault Detection for Windows

Atlantic Technological University Ireland

Andrew Fox · G00417373 · Supervisor: Brian O'Shea

02

The Problem

Windows Task Manager

Current load only. No historical Data logging, no fault detection.

HWMonitor / HWiNFO

Deep sensor readings. Raw numbers, no interpretation.

NZXT CAM

Consumer UI. Requires a cloud account.

MSI Afterburner

Logs GPU data. No diagnostic capability.

THE GAP

What's missing:

No tool combines persistent local storage, automated fault detection, and plain-language reporting.

Technical users want detailed data and export capability.

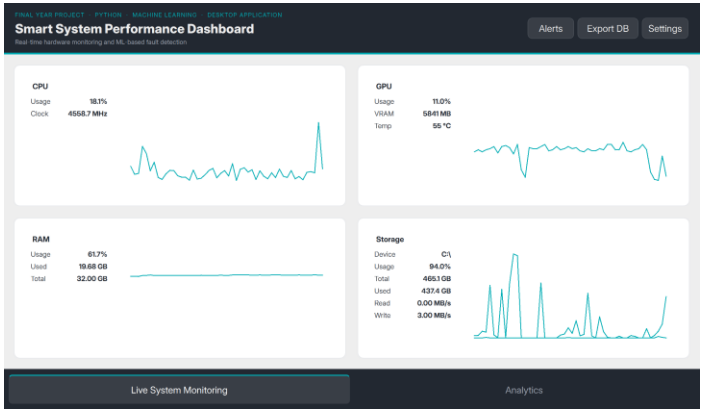
Everyday users just want to know: "Is something wrong with my PC?"

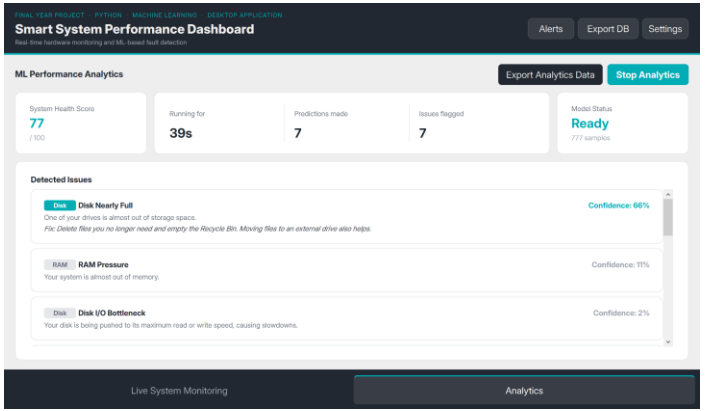
Existing tools display data - none of them diagnose it.

03

The Solution

- A Windows desktop application built entirely in Python
- Monitors CPU, RAM, GPU & Disk in real time, every second
- Everything stored locally in a SQLite database
- Runs a trained ML model to automatically detect hardware faults
- Reports findings in plain language with a suggested fix
- Fully offline - no cloud, no account, no admin rights required





Requirements & Success Criteria

REQUIREMENTS

FR1 Collect CPU, RAM, GPU & Disk metrics every 1 second

FR2 Store all telemetry in a local SQLite database

FR3 Run ML classifier every 5s on latest telemetry window

FR4 Display real-time health score from 0 to 100

FR5 Configurable graph refresh rate, saved to settings file

FR6 CSV export of analytics session results and full telemetry

FR7 Runs on machines with no dedicated NVIDIA GPU

SUCCESS CRITERIA ✓

SC1 ✓
No UI freezing
28-46ms cycles, GUI thread never blocked

SC2 ✓
Metric accuracy
Values crosschecked with Windows Task Manager

SC3 ✓
Persistent storage
All telemetry stored & queryable by session

SC4 ✓
ML fault detection
12 labels + confidence scores

Project Management

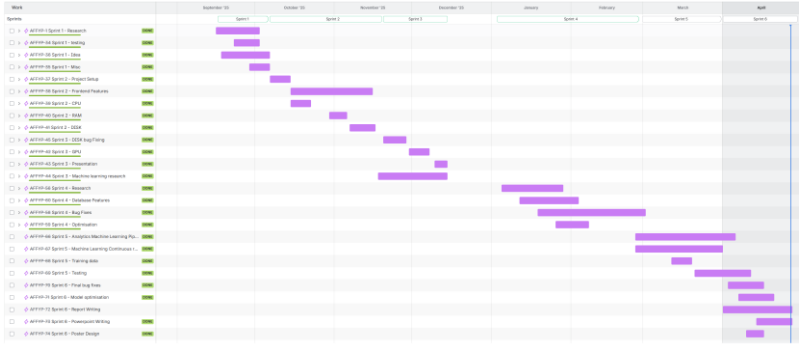
Scrum methodology with weekly in-class stand-ups

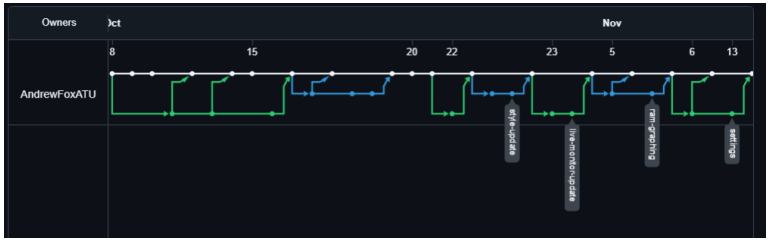
Jira board maintained across both semesters

OneNote used for problem logs, solutions & design decisions

Regular supervisor meetings throughout development

GitHub used throughout the project for version control





System Overview

01

Collector

QThread · 1-second loop

- Polls CPU, RAM, GPU & Disk every second
- Uses psutil, pywin32 & pynvml - windows APIs
- Tracks dropped metrics if a collector fails
- Runs in background thread never blocks the UI

02

Storage

SQLite · WAL Mode

- Writes every sample to SQLite instantly
- WAL mode allows reads & writes at the same time
- Each machine gets a unique UUID automatically
- 10 tables connected with foreign keys

03

Analytics

ML Inference · 5s cycle

- Waits for 100 samples before first prediction
- Builds a 36-feature vector from the last 10 samples
- Random Forest predicts 12 fault types simultaneously
- Calculates a health score 0–100 from results

04

Dashboard

PyQt6 · stylised GUI

- Live scrolling graphs for CPU, RAM, GPU & Disk
- Analytics panel shows detected faults + fix advice
- Export raw telemetry or analytics summary as CSV
- Settings saved between sessions

Fully offline · No cloud · No account · All telemetry stays on the user's machine

Dashboard GUI & Health Score

Live Monitor

2×2 pyqtgraph scrolling graphs showing CPU, GPU, RAM, Storage metrics
50-point rolling window · 20 FPS hover tooltip

Analytics Panel

12 issue cards sorted by confidence
Plain-language descriptions + suggested fixes

Alerts Log

Unique alert per fault - no duplicate entries
Timestamp, component, confidence

Export Options

Export DB → full raw telemetry CSV
Export Analytics → session summary + issues

HEALTH SCORE FORMULA

$$100 - (\text{penalty} \times \text{confidence})$$

5 — Critical	gpu_overheating	60 pts
4 — Serious	cpu_throttle · ram_memory_leak · disk_full	35 pts
3 — Warning	cpu_bottleneck · ram_pressure · +4 more	15 pts
2 — Minor	cpu_high_load · disk_bottleneck	5 pts

Penalty scales with model confidence.
Multiple simultaneous faults accumulate.

Machine Learning Pipeline

01

Feature Engineering

36 features · 10-sample window

- 15 point-in-time readings per sample
- 21 rolling stats - mean, std & slope
- Captures trends not just current values
- Built fresh every 5 seconds

02

Label Generation

Rule-based thresholds

- 12 fault labels defined by threshold rules
- No manual labelling required
- Rules based on real hardware behaviour
- Applied automatically across entire dataset

03

Training Data

Real + Synthetic

- Collected from 3 physical machines over multiple sessions
- Synthetic data generated to cover fault edge cases
- Exported directly from the Storage Service
- Combined into one labelled CSV for training

04

Model Training

200 trees

- MultiOutputClassifier predicts all 12 labels at once
- class_weight=balanced handles uneven data
- 80/20 train/test split, random seed for reproducibility
- Trained once, saved and loaded at runtime

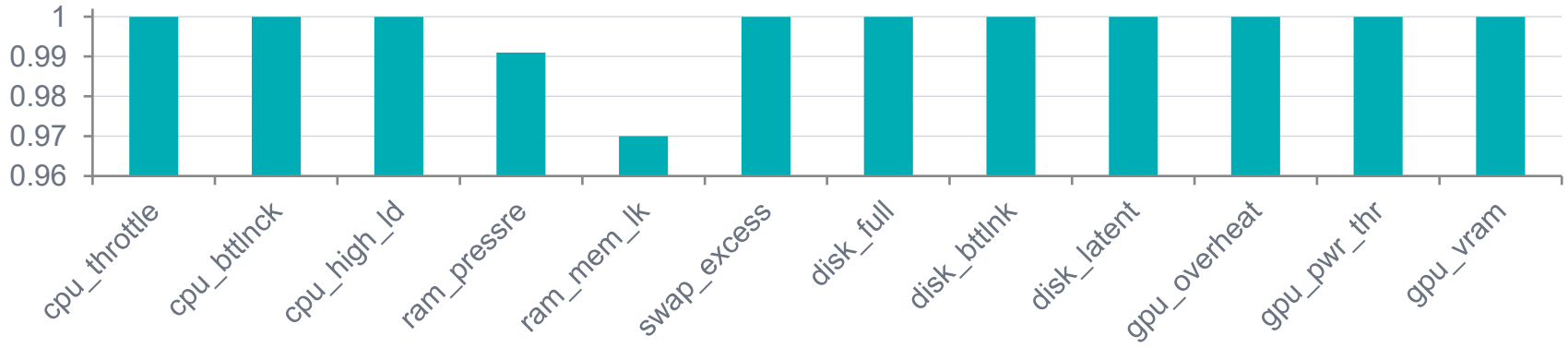
CPU: thermal_throttle · bottleneck · sustained_high_load RAM: pressure · memory_leak · excessive_swap
Disk: full · bottleneck · high_latency GPU: overheating · power_throttle · vram_pressure

Model Results

AVG F1 SCORE · ALL 12 LABELS

0.98

12	fault types - CPU, RAM, Disk & GPU
12,000 +	real telemetry samples (3 different machines)
8,800 +	synthetic samples (12 scenario builders)
80 / 20	train / test split



Testing

AUTOMATED TESTS

80+

test_collectors.py

- Tests every metric key, value range and edge case
- Handles PDH absent, GPU absent and PermissionError

test_storage.py

- Fully isolated in-memory SQLite - no filesystem needed
- Tests host registration, session lifecycle and data integrity

test_labels.py

- Verifies all 12 labels fire at threshold
- Tests healthy baseline and combined fault conditions

test_model.py

- End-to-end pipeline across 9 fault scenario windows
- Verifies output structure and missing-feature robustness

Multi-machine: Laptop (Win 11 + GPU) · Desktop (Win 10 + GPU) · School PC (Win 11, no GPU) - all pass

Key Challenges

CHALLENGE 01

GPU Collector Bottleneck

- GUI freezing 0.5–1.0s on every collection cycle
- GPU collector taking 344ms - others under 8ms
- Cause: pynvml re-initialising on every call + slow PCIe metrics
- Fix: moved init to module level and removed PCIe metrics - cycles dropped to 22ms

CHALLENGE 02

CPU Frequency Inaccuracy

- psutil returned a fixed 2304MHz regardless of load
- Cause: Windows reports base clock, not real-time frequency
- Fix: switched to Windows PDH Actual Frequency counter
- Windows 10 fallback: % Processor Performance × base clock

CHALLENGE 03

DB Session Overlap Bug

- Previous session's disk data appearing in new sessions
- Cause: wrong variable passed as partition_id on insert
- Fix: one-line variable correction

CHALLENGE 04

CPU Temperature - Deferred

- No universal Windows API for CPU die temperature
- WMI needs admin · LibreHardwareMonitor needs kernel driver
- Deferred rather than shipping a silently-wrong implementation

Conclusion

- Fully local Windows app - monitors, detects faults, explains them in plain language
- Four services: Collector, Storage, Analytics, Dashboard
- 0.98 avg F1 across 12 fault types - trained on 20,000+ samples
- Collection cycle reduced from 468ms to 22–38ms
- All 7 requirements met · all 4 success criteria verified
- 80+ automated tests across 3 machines

The Smart System Performance Dashboard demonstrates how a well-designed modular architecture can combine hardware systems programming, database design, and machine learning in a single cohesive local application - making hardware diagnostics accessible to every user, regardless of technical background.